QALight

Frontend-розробка

Заняття 13. Функції

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Визначення функції, області видимості, аргументи, директива return.
Функціональні вирази, функції зі стрілками.
Створення власних функцій

Функції

Визначення функції, області видимості, аргументи, директива return

JavaScript функції

Теорія: JavaScript функції

Функція - це засіб для структурування великих програм, зменшення повторень, та ізолювання підпрограм одну від іншої.

Ключове слово **function** йде першим, потім - **ім'я функції**, потім - **список параметрів, якщо є,** у дужках (розділений комами) і, нарешті, код функції - «**тіло функції**», у фігурних дужках.

```
function ім'я функції(список параметрів) {
  // тіло функції
};
```

Оголошення (декларація) функцій

Теорія: JavaScript функції

Щоб створити функцію треба її оголосити/
Приклад оголошення функції:

```
function square(number) {
return number * number;
}
```

Функція містить **блок кода**, який можна викликати у будь-якій частині програми.

Код функції **виконується лише після явного виклику функції.**

Функцію можна викликати до її визначення. Виклик square(22) виконує код функції.

```
square(22)
```

Параметри функції

Теорія: JavaScript функції

Ми можемо передати в функцію довільні дані використовуючи **параметри**.

В наступному прикладі, функція має два параметри: from і text.

```
function showMessage(from, text) { // параметри: from, text
    console.log(from + ': ' + text);
}
showMessage('Анна', 'Привіт!'); // Анна: Привіт! (*)
showMessage('Анна', "Як справи?"); // Анна: Як справи? (**)
```

Під час виклику функції з цими параметрами, в рядках (*) та (**) відбувається копіювання значень параметрів в **локальні змінні** from та text. Ці змінні використовує функція.

Значення задаються при виклику функції

Теорія: JavaScript функції

Маємо змінну from, яку передаємо в функцію, що змінює значення from, проте ці зміни не видно назовні, тому що функція завжди отримує копію значення:

```
function showMessage(from, text) {
    from = '*' + from + '*'; // прикрашаємо "from"
    console.log( from + ': ' + text );
}

let from = "Анна";
showMessage(from, "Привіт"); // *Анна*: Привіт

// значення "from" те саме, функція змінила локальну копію
console.log( from ); // Анна
```

Коли значення передається як параметр функції, то його ще називають аргумент.

Параметри функції – змінні, значення яких задаються при виклику функції.

Локальні змінні

Теорія: JavaScript функції

Змінні, що створені всередині функції (включаючи параметри), називають **локальними змінними всередині цієї функції**.

Функція може містити локальні змінні, що визначені через **var/let**. Змінна, оголошена усередині функції, **видима лише всередині цієї функції**:

```
function showMessage() {
    var message = 'Привіт!'; // локальна змінна
    alert( message );
}
showMessage(); // 'Привіт!'
```

Змінна недоступна поза функцією

```
alert( message ); // <-- Error! The variable is local to the function
```

Зовнішні змінні

Теорія: JavaScript функції

Змінні, що задані за межами тіла функції, назвають **глобальними**. Отримати доступ до глобальних змінних можна всередині функції, якщо їх **не було перевизначено**.

```
var userName = 'Jhon';
function showMessageUser() {
    var message = 'Привіт ' + userName;
    alert(message);
}
showMessageUser(); // Привіт Jhon
```

Функція має повний доступ до зовнішньої змінної. Вона також може змінити його.

```
function showMessageUser() {
    userName = 'Bob';
    var message = 'Привіт ' + userName;
    alert(message);
}
showMessageUser(); // Привіт Bob
```

Хорошою практикою вважається мінімізація використання глобальних змінних.

Функція завжди віддасть перевагу локальним змінним

Теорія: JavaScript функції

Зовнішня змінна використовується тоді, коли немає локальної.

Якщо всередині функції є змінна з таким самим ім’ям, то вона перекриває зовнішню. Наприклад, наступний код використовує локальну змінну userName. Зовнішня ігнорується.

```
let userName = 'Іван'; // оголошення зовнішньої змінної

  function showMessage() {
     let userName = "Богдан"; // оголошення локальної змінної
     let message = 'Привіт, ' + userName; // Богдан
     console.log(message);
  }

  showMessage();
```

Функція завжди віддасть перевагу локальним змінним

```
  console.log( userName ); // Іван, без змін, функція не змінила глобальну змінну
```

Аргументи за замовчуванням

Теорія: JavaScript функції

Ми можемо передавати довільні дані функціям за допомогою **аргументів**. Функція може мати **будь-яку кількість аргументів**. Аргумент - це значення, яке передається функції під час її виклику. Якщо при виклику функції аргументу не призначено значення – він вважається **undefined**.

```
function showMessage(from, text) {
        if (text === undefined) {   text = 'текст не призначено';   }
        alert( from + ": " + text );
}
```

У якості аргумента за замовчуванням можна використовувати результат **оператора ||**:

```
function showMessage(from, text) {   text = text || 'текст не призначено';   }
```

Аргумент відсутній, якщо його значення в логічному контексті дорівнює **false**.

Виклик функції без аргументів

Теорія: JavaScript функції

Якщо викликати функцію без аргументів, тоді відповідні значення стануть undefined. Наприклад, функцію showMessage(from, text) можна викликати з одним аргументом:
showMessage("Анна");

Помилки не виникне. Такий виклик виведе "*Анна*: undefined". Оскільки значення для змінної text не задане, воно стане undefined. Ми можемо задати так зване “**типове**” значення параметра, яке використовуватиметься, якщо не задати аргумент. Для цього потрібно написати значення через =:

```
function showMessage(from, text = "текст не задано") {
    console.log( from + ": " + text );
}

showMessage("Анна"); // Анна: текст не задано
```

Тепер, якщо параметр text не задано, його значення стане "текст не задано".

Типове значення

Теорія: JavaScript функції

Типове значення також додається, якщо параметр існує, але суворо дорівнює undefined, наприклад:

```
showMessage("Анна", undefined); // Анна: текст не задано
```

Тут "текст не задано" це рядок, проте це може бути складніший вираз, який обчислюється і присвоюється лише якщо параметр відсутній. Отож, такий варіант теж можливий:

```
function showMessage(from, text = anotherFunction()) {
        // anotherFunction() виконується лише якщо `text` не задано
        // результат виконання цієї функції присвоїться змінній `text`
}
```

В JavaScript, типовий параметр обчислюється кожного разу, коли викликається функція без відповідного параметру.

Альтернативні типові параметри

Теорія: JavaScript функції

Інколи виникає необхідність присвоїти типові значення для змінних під час виконання функції, а не під час її оголошення. Під час виконання функції, ми можемо перевірити, чи параметр надано, порівнюючи його з undefined:

```
function showMessage(text) {
 if (text === undefined) { // якщо параметр відсутній
   text = 'порожнє повідомлення';
 }

 alert(text);
}showMessage(); // порожнє повідомлення

// …Або ми можемо використати оператор ||:

function showMessage(text) {
 // якщо text не задано (значення `undefined`) або `null`, тоді присвоїти рядок 'порожньо'
 text = text || 'порожньо';
 ...
}
```

Альтернативні типові параметри

Теорія: JavaScript функції

Сучасні рушії JavaScript підтримують оператор null-злиття ??. Його краще використовувати, коли “майже false” значення, типу 0, мають вважатися за “нормальні”:

```
function showCount(count) {
 // якщо count має значення undefined чи null, показати "невідомо"
 console.log(count ?? "невідомо");
}

showCount(0); // 0
showCount(null); // невідомо
showCount(); // невідомо
```

Тернарний оператор

Практика: JavaScript функції

Тернарний оператор - це представлення блоку **if/else** у згорнутому форматі. Загальний формат тернарного оператора: **condition ? expr1 : expr2.** Докладніше: MDN
Якщо condition істинна, оператор виконує expr1; інакше - expr2. Наприклад, щоб відобразити інше повідомлення на основі значення змінної isMember, ви можете використати цей оператор:

```
let isMember = true;
let message = isMember ? 'Welcome Back!' : 'You need to login'; // 'Welcome Back'
```

Іншим корисним методом використання тернарного оператора є умовне виконання функції

```
 function memberOpen(){
     console.log("open");
}
 function memberClose(){
     console.log("close");
}
 let isMember = true;  (isMember) ? memberOpen() : memberClose();
```

Також можна запускати функції за допомогою тернарного оператора:

```
const runFirst = true;
runFirst ? first() : second();
```

Оператор нульового об’єднання

Практика: JavaScript функції

Оператор **nullish coalescing** - **нульового об’єднання** (**??**) - це логічний оператор, який повертає правий операнд, якщо його лівий операнд є **null** або **undefined**, а в інших випадках повертає його лівий операнд. Це можна розглядати як особливий випадок логічного оператора **АБО** (||), який повертає правий операнд, якщо лівий операнд має будь-яке хибне значення, а не лише нульове або невизначене. Оператор **nullish coalescing** використовується для повернення правої частини кожного разу, коли ліва частина нульова або невизначена:

```
// Is set to 0
let x = 0 ?? "hello";
// Is set to goodbye
let y = undefined ?? "goodbye";
// Is set to hello
let z = null ?? "hello";
// Is set to false
let a = false ?? "goodbye";
```

Синтаксис оператора нульового об’єднання

Оператор об’єднання з null записується як два знаки питання ??.
Результатом a ?? b буде:
- a, якщо a визначене,
- b, якщо a не визначене.

Оператор ?? повертає перший аргумент, якщо він не null/undefined. Інакше, другий.
Оператор об’єднання з null не є абсолютно новим. Це просто хороший синтаксис, щоб отримати перше “визначене” значення з двох.

Ми можемо переписати вираз result = a ?? b, використовуючи оператори, які ми вже знаємо:

```
result = (a !== null && a !== undefined) ? a : b;
```

Приклад оператора нульового об’єднання

Наприклад, показуємо значення у змінній user, якщо її значення не null/undefined, інакше – показуємо Анонімний:

```
let user;
console.log(user ?? "Анонімний"); // Анонімний (user є undefined)
```

Ми також можемо використовувати послідовність з ??, щоб вибрати перше значення зі списку, яке не є null/undefined. Скажімо, у нас є дані користувача в змінних firstName, lastName або nickName. Всі вони можуть бути не визначені, якщо користувач вирішив не вводити значення.
Ми хотіли б показати ім’я користувача, використовуючи одну з цих змінних, або показати “Анонімний”, якщо всі вони null/undefined:

```
let firstName = null;
let lastName = null;
let nickName = "Суперкодер";

// показує перше визначене значення:
console.log(firstName ?? lastName ?? nickName ?? "Анонімний"); // Суперкодер
```

Оператор ||

Практика: JavaScript функції

Оператор АБО || може бути використаний таким же чином, як ??.

```
let firstName = null;
let lastName = null;
let nickName = "Суперкодер";

// показує перше істинне значення:
console.log(firstName || lastName || nickName || "Анонімний"); // Суперкодер
```

Оператор || не розрізняє, чи значення false, 0, порожній рядок "" чи null/undefined. Всі вони однакові – хибні значення. Якщо будь-яке з них є першим аргументом ||, тоді ми отримаємо другий аргумент як результат.

Однак на практиці, ми хочемо використовувати типове значення лише тоді, коли змінна null/undefined. Тобто, коли значення дійсно невідоме/не встановлене.

Наприклад, розгляньте це:

```
let height = 0;

alert(height || 100); // 100
alert(height ?? 100); // 0
```

height || 100 перевіряє чи має змінна height має хибне значення, і 0 – це дійсно хибне значення.
отже, результатом || є другий аргумент, 100.
height ?? 100 перевіряє змінну height, чи вона null/undefined, і це не так,
отже, результат – height “як є”, тобто 0.

На практиці нульова висота часто є дійсним значенням, яке не слід замінювати на типове значення. Отже, ?? робить саме те, що треба.

Порівняння ?? з ||

Практика: JavaScript функції

Важлива різниця між ?? і || полягає в тому, що:
|| повертає перше істинне значення.
?? повертає перше визначене значення.
На практиці, ми хочемо використовувати типове значення лише тоді, коли змінна null/undefined.
Тобто, коли значення дійсно невідоме/не встановлене.

```
let height = 0;
alert(height || 100); // 100
alert(height ?? 100); // 0
```

height || 100 перевіряє чи має змінна height має хибне значення, і 0 – це дійсно хибне значення.
отже, результатом || є другий аргумент, 100.
height ?? 100 перевіряє змінну height, чи вона null/undefined, і це не так,
отже, результат – height “як є”, тобто 0.

На практиці нульова висота часто є дійсним значенням, яке не слід замінювати на типове значення.

Директива return

Теорія: JavaScript функції

Для повернення значення використовується директива **return**. Директива може знаходитись у будь-якому місці функції.

Коли директива **return** отримує управління – функція завершує свою роботу і управління передається на рівень вище.

```
function calcD(a, b, c) {
      return b*b - 4*a*c;
  }
var test = calcD(-4, 2, 1);
alert(test); // 20
```

Директива return може бути в будь-якому місці функції. Коли виконання досягає цієї директиви, функція зупиняється, і в код, який викликав цю функцію, повертається значення.

Кілька директив return

Теорія: JavaScript функції

В одній функції може бути багато директив **return**. Наприклад:

```
function checkAge(age) {
  if (age >= 18) {
                return true;
  } else {
                return confirm('Do you have permission from your parents?');
  }

function checkAge(age) {
  if (age >= 18) {
                return true;
  } else {
                return confirm('Do you have permission from your parents?');
  }
```

Негайний вихід із функції

Практика: JavaScript функції

Можна використовувати return без значення. Це призводить до негайного виходу з функції.
Наприклад:

```
function showMovie(age) {
 if ( !checkAge(age) ) {
   return;
 }
alert( "Showing you the movie" ); // (*)
 // ...
}
```

Функція з порожнім поверненням або без нього повертає undefined
Порожнє повернення теж саме, що return undefined:

```
function doNothing() {
 return;
}
alert( doNothing() === undefined ); // true
```

Функції

Функціональні вирази, функції зі стрілками.
Створення власних функцій

Функціональні вирази

Практика: JavaScript функції

Синтаксис, що ми раніше використовували, називається Оголошення Функції (Function Declaration):

```
ffunction sayHi() {
 alert( "Привіт" );
}
```

Існує й інший синтаксис для створення функції, що називають Функціональним Виразом (Function Expression). Він дозволяє створювати функцію всередині будь-якого виразу.

```
let sayHi = function() {
 alert( "Привіт" );
};
```

Оскільки створення функції відбувається в контексті присвоєння виразу (праворуч від =), це називається Функціональним виразом (Function Expression).
Після ключового слова function немає назви функції. Для функціональних виразів це допустимо.

Анонімні функції

Практика: JavaScript функції

Функцію можна створити та відразу ж викликати або запланувати її на відтерміноване виконання. Такі функції ніде не зберігаються, тому вони залишаються **анонімними**.

Функціональний вираз - анонімна функція, тобто вона не має імені, яка призначається змінній:

```
let myFunction = function() {
 // do something
};
```

Функціональні вирази потрібно визначити перед тим, як їх викликати. Наприклад:

```
let add = function (a, b) {
 return a + b;
};
console.log(add(1, 2));
```

Функція – це значення

Практика: JavaScript функції

Немає значення, яким чином створено функцію, функція – це завжди значення, тому ми можемо поводитись з нею, як і з іншими значеннями. Звичайно, функція - особливе значення, у тому сенсі, що ми можемо здійснити її виклик за допомогою дужок: sayHi().

Але це все-таки значення. Тому ми можемо працювати з нею, як і з іншими значеннями. Скажімо, ми можемо скопіювати функцію в іншу змінну:

```
function sayHi() {   // (1) створюємо
 alert( "Привіт" );
}

let func = sayHi;    // (2) копіюємо
func(); // Привіт     // (3) викликаємо копію (працює!)
sayHi(); // Привіт    //     ось так теж спрацює (а чому ні?)
```

Функціональний вираз містить крапку з комою ; в кінці

Функціональний Вираз містить крапку з комою ; в кінці, а Оголошення Функції — ні:

```
function sayHi() {
 // ...
}

let sayHi = function() {
 // ...
};
```

Крапка з комою ; не є частиною синтаксису функції. Вона є частиною інструкції, тому її рекомендується ставити в кінці інструкції.

Функції зі стрілками

Практика: JavaScript функції

Функції зі стрілками схожі на звичайні функції, за винятком кількох речей. Перш за все, синтаксис для написання функції зі стрілкою є компактнішим, ніж звичайна функція:

```
Regular function
function getPrice(cost, quantity) {
 return cost * quantity;
}

Arrow function
(cost, quantity) => cost * quantity;
```

Це створює функцію, яка приймає аргументи cost, quantity, потім обчислює вираз у правій частині з їх використанням і повертає результат.

Аргументи функцій зі стрілками

Практика: JavaScript функції

ліворуч від => аргументи, а справа — окремий вираз, що певним чином їх використовував.

Якщо у нас є тільки один аргумент, то дужки навколо параметрів можна опустити, що робить функції зі стрілками ще коротшими. Наприклад:

```
let double = n => n * 2;
alert( double(3) ); // 6
```

Функції зі стрілками можна використовувати так само, як і вирази функцій. Наприклад, щоб динамічно створити функцію:ґ

```
let sayHi = () => alert("Hello!");
sayHi();
```

Багаторядкові стрілкові функції

Практика: JavaScript функції

Іноді нам потрібна більш складна функція з кількома виразами та операторами. У цьому випадку ми можемо взяти їх у фігурні дужки. Основна відмінність полягає в тому, що фігурні дужки вимагають оператор return, щоб повернути значення (так само, як і звичайна функція).

```
let sum = (a, b) => {  // the curly brace opens a multiline function
 let result = a + b;
 return result; // if we use curly braces, then we need an explicit "return"
};

alert( sum(1, 2) ); // 3
```

Функціональний Вираз проти Оголошення Функції

Практика: JavaScript функції

Функцію можна сприймати як дію. Ми можемо передавати її від змінної до змінної і викликати коли заманеться.

Ключові відмінності між Оголошенням Функції та Функціональним Виразом.

- Оголошення Функції: функція оголошується окремою конструкцією “function…” в основному потоці коду.
- Функціональний Вираз: функція створюється як частина іншого виразу чи синтаксичної конструкції.

Функціональний Вираз буде створено тільки тоді, коли до нього дійде виконання і тільки після цього він може бути використаний.

Щойно потік виконання досягне правої частини у присвоєнні let sum = function…, функцію буде створено і з цього моменту її можна буде використати (присвоїти змінній, викликати тощо).

Область видимості

Практика: JavaScript функції

Ще однією особливістю Оголошення Функції є її блокова область видимості. У суворому режимі, якщо Оголошення Функції знаходиться в блоці {...}, то функція доступна всередині блоку. Але не зовні.

```
let age = prompt("Скільки вам років?", 18);

// оголошуємо функцію відповідно до умови
if (age < 18) {
 function welcome() {
   alert("Привіт!");
 }
} else {
 function welcome() {
   alert("Вітання!");
 }
}
// ...спробуємо викликати функцію
welcome(); // помилка в суворому режимі (ReferenceError: welcome is not defined)
```

Оголошення Функції доступне тільки всередині блоку, що його містить.

Область видимості

Практика: JavaScript функції

Правильніше було б використати Функціональний Вираз і присвоїти welcome змінній, що оголошена поза блоком if і доступна для нас.

```
let age = prompt("Скільки вам років?", 18);
let welcome;
if (age < 18) {

 welcome = function() {
   alert("Привіт!");
 };
} else {

 welcome = function() {
   alert("Вітання!");
 };

}
welcome(); // тепер все гаразд
```

Рекурсивна функція

Практика: JavaScript функції

Рекурсивна функція - це функція, яка викликає сама себе десь у тілі функції.

```
function recursiveFunc() {
 recursiveFunc()
}
```

Функція буде повторювати виклик, доки не буде досягнуто бажаного результату. Базова умова дозволяє функції припинити виклик самої себе.

```
function doSomething(n) {
 if(n === 0) {
 console.log("TASK COMPLETED!")
 return
 }
 console.log("I'm doing something.")
 doSomething(n - 1)
}
doSomething(3)
```

Рекурсивна функція pow

Практика: JavaScript функції

Напишемо функцію pow(x, n), що підносить x до натурального ступеня n.

```
function pow(x, n) {
  if (n == 1) {
    return x;
  } else {
    return x * pow(x, n - 1);
  }
}
alert( pow(2, 3) ); // 8
```

Коли функція pow(x, n) викликається, виконання розділяється на дві гілки:
Якщо n == 1, то все просто. Ця гілка називається базою рекурсії, оскільки вона негайно дає очевидний результат: pow(x, 1) дорівнює x.
Інакше ми можемо представити pow(x, n) як x * pow(x, n). Що у математиці можна написати xn = x * xn-1. Ця гілка – це крок рекурсії: ми зводимо задачу до більш простої дії (множення на x) і до більш простої аналогічної задачі (pow з меншим n).

Рішення з використанням рекурсії

Практика: JavaScript функції

Рекурсивне рішення, як правило, коротше за ітераційне.

Ми можемо переписати те ж саме, використовуючи умовний оператор ? замість if, щоб зробити pow(x, n) більш лаконічним і зберегти легкість читання:

```
function pow(x, n) {
 return (n == 1) ? x : (x * pow(x, n - 1));
}
```

Максимальна глибина рекурсії

Практика: JavaScript функції

Максимальна кількість вкладених викликів (включаючи перший) називається глибина рекурсії. Максимальна глибина рекурсії обмежена рушієм JavaScript. Кількість вкладених викликів, на яку точно можна розраховувати – 10 000. Деякі рушії дозволять працювати з ще більшою глибиною. Але навіть більшість з них можуть осилити не більше 100 000 викликів. Є багато автоматичних оптимізацій, які допомагають уникнути переповнення стеку викликів (“оптимізація хвостових викликів”, англ. “tail call optimizations”), але вони підтримуються далеко не всюди і працюють тільки в простих випадках.

Це обмежує застосування рекурсії, але вона все ще залишається дуже широко поширеною. Є багато завдань, де рекурсивний спосіб мислення дає простіший код, який легше підтримувати.

Контекст виконання

Практика: JavaScript функції

Інформація про процес виконання викликаної функції зберігається у контексті виконання.

Контекст виконання – це внутрішня структура даних, яка містить деталі про виконання функції: де зараз керуючий потік, поточні змінні, значення this (ми не використовуємо його тут) і кілька інших внутрішніх деталей.

Один виклик функції має рівно один контекст виконання, пов’язаний з ним.

Коли функція робить вкладений виклик, відбувається наступне:

- Поточна функція зупиняється.
- Контекст виконання, пов’язаний з нею, запам’ятовується в спеціальній структурі даних, що називається стек контекстів виконання.
- Виконуються вкладені виклики і для кожного з них створюється свій контекст виконання.
- Після закінчення, старий контекст виконання витягується з стека, і зовнішня функція відновлюється з того місця, де вона зупинилася.

Факторіал з натурального числа

Практика: JavaScript функції

Факторіал з натурального числа – це число, помножене на "число мінус один", потім на "число мінус два" і так до 1. Факторіал n позначається як n!

Ми можемо написати визначення факторіалу наступним чином:

n! = n * (n - 1) * (n - 2) * ...*1

Значення факторіалів для різних n:

1! = 1

2! = 2 * 1 = 2

3! = 3 * 2 * 1 = 6

4! = 4 * 3 * 2 * 1 = 24

5! = 5 * 4 * 3 * 2 * 1 = 120

Завдання полягає в тому, щоб написати функцію factorial(n), яка обчислює n! за допомогою рекурсивних викликів.

```
alert( factorial(5) ); // 120
```

P.S. Підказка: n! може бути записане як n * (n-1)!. Наприклад: 3! = 3*2! = 3*2*1! = 6

Обчислити факторіал натурального числа за допомогою рекурсії в JavaScript можна за допомогою простої функції, яка викликає сама себе з меншим значенням, доходячи до базового випадку 1. [1] (https://uk.javascript.info/recursion), [2] (https://romul.name/blog/rekursiya-v-javascript/)

Факторіал з натурального числа

Практика: JavaScript функції

Обчислити факторіал натурального числа за допомогою рекурсії в JavaScript можна за допомогою простої функції, яка викликає сама себе з меншим значенням, доходячи до базового випадку 1. [1] (https://uk.javascript.info/recursion), [2] (https://romul.name/blog/rekursiya-v-javascript/), [3] (https://medium.com/@t4rg4rnen/%D0%B8%D0%B7%D1%83%D1%87%D0%B5%D0%BD%D0%B8%D0%B5-%D0%B8-%D0%BF%D0%BE%D0%BD%D0%B8%D0%BC%D0%B0%D0%BD%D0%B8%D0%B5-%D1%80%D0%B5%D0%BA%D1%83%D1%80%D1%81%D0%B8%D0%B8-%D0%B2-javascript-fe44fc02410b)Приклад кодуjavascriptfunction factorial(n) {

```
  // Базовий випадок: якщо n дорівнює 1 або 0, повертаємо 1
  if (n <= 1) {
    return 1;
  }
  // Рекурсивний випадок: множимо n на факторіал (n - 1)
  return n * factorial(n - 1);
}
console.log(factorial(5)); // 120
```

Use code with caution.Також можна написати цей код коротше за допомогою тернарного оператора:javascriptconst factorial = (n) => (n <= 1 ? 1 : n * factorial(n - 1));

```
console.log(factorial(5)); // 120
```

Use code with caution. Більше інформації та розбір інших способів можна знайти в підручнику з JavaScript. [1] (https://uk.javascript.info/recursion) Якщо хочете, я можу показати: Як написати цю ж функцію через цикл (ітеративно), Як додати перевірку на помилки (для від'ємних чисел чи нецілих значень), Що з цього вас цікавить?

Функція, що обчислює n! за допомогою рекурсії

Практика: JavaScript функції

Обчислити факторіал натурального числа за допомогою рекурсії в JavaScript можна за допомогою простої функції, яка викликає сама себе з меншим значенням, доходячи до базового випадку 1.

```
function factorial(n) {
 // Базовий випадок: якщо n дорівнює 1 або 0, повертаємо 1
 if (n <= 1) {
   return 1;
 }
 // Рекурсивний випадок: множимо n на факторіал (n - 1)
 return n * factorial(n - 1);
}

console.log(factorial(5)); // 120
```

Також можна написати цей код коротше за допомогою тернарного оператора:

```
const factorial = (n) => (n <= 1 ? 1 : n * factorial(n - 1));

console.log(factorial(5)); // 120
```

Послідовність чисел Фібоначчі

Практика: JavaScript функції

Послідовність Фібоначчі — це числовий ряд, у якому кожне наступне число є сумою двох попередніх. Числа цієї послідовності виглядають так: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144 тощо.

Формула: \(F_n = F_{n-1} + F_{n-2}\), де початкові значення зазвичай $F_0 = 0$ та $F_1 = 1$.

Закономірність Фібоначчі часто зустрічається в природі, наприклад, у розташуванні пелюсток квітів, насіння соняшника чи листя на стеблі.

Відношення сусідніх чисел Фібоначчі наближається до числа 1,618, яке відоме як золотий перетин.

Ряд Фібоначчі використовують у математиці, програмуванні та технічному аналізі на фінансових ринках.

Перші два числа: 1, потім 2(1+1), потім 3(1+2), 5(2+3) і так далі: 1, 1, 2, 3, 5, 8, 13, 21....

Функція для отримання (n)-го числа Фібоначчі

Практика: JavaScript функції

Функція для отримання \(n\)-го числа Фібоначчі

```
function fib(n) {
 // Базовий випадок
 if (n <= 1) {
   return n;
 }
 // Рекурсивний виклик
 return fib(n - 1) + fib(n - 2);
}

console.log(fib(6)); // Виведе: 8 (послідовність: 0, 1, 1, 2, 3, 5, 8)
```

Виведення цілої послідовності до (n)-го елемента

Практика: JavaScript функції

Щоб отримати масив із перших \(n\) чисел, можна викликати рекурсивну функцію для кожного індексу в циклі або за допомогою рекурсивного збирання масиву:

```
function fib(n) {
 return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

function getFibonacciSequence(length) {
 const result = [];
 for (let i = 0; i < length; i++) {
   result.push(fib(i));
 }
 return result;
}

console.log(getFibonacciSequence(7)); // Виведе: [0, 1, 1, 2, 3, 5, 8]
```

Чиста рекурсія для Фібоначчі працює повільно при великих n (складність \(O(2^n)\)), оскільки робить багато однакових повторних обчислень.

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua